

Orin NX Software Guide

Every program on the board: what it does, what it is really measuring, and how to build your own software on top of it.

COMPUTE	CARRIER	CAMERA	AUTOPILOT	REACH IT AT
Orin NX 16 GB	Seeed A603	Sony IMX477	Betaflight / MSP	dcl@192.168.55.1

Who this is for: participants who are past the quick start (`QUICKSTART.md`) and now want to know what every program on the board does, what it is really measuring, and how to build their own software on top of it.

Everything described here is already installed. Nothing in this document needs `pip` , a network connection, or root — except where it says so explicitly.

1. The stack, layer by layer

Work from the bottom. Almost every confusing symptom on this board is a lower layer failing in a way an upper layer reports badly.

LAYER	WHAT IT IS	HOW YOU INSPECT IT
Sensor	IMX477, raw Bayer, 10-bit, on a 15-pin CSI-2 connector	<code>i2cdetect -y -r 2</code> — the sensor must ACK its address
Device tree	Tells the kernel a camera exists and how it is wired	<code>ls /proc/device-tree/bus@0/i2c@3180000/</code>
Driver	The imx477 kernel driver binds the I ² C client and drives reset and mode registers	<code>dmesg grep -i imx477</code>
Capture	NVCSI receives the MIPI lanes, VI writes frames to memory	<code>/dev/video0</code> exists
V4L2	The raw interface. Bayer bytes, no processing	<code>v4l2-ctl --list-formats-ext</code>
Argus / ISP	Debayer, auto-exposure, auto-white-balance, noise reduction	<code>nvarguscamerasrc</code> ; <code>systemctl status nvargus-daemon</code>
GStreamer	Pipelines that glue the above to your code	<code>gst-inspect-1.0 nvarguscamerasrc</code>
OpenCV	JetPack's build, with GStreamer support	<code>python3 -c 'import cv2; print(cv2.getBuildInformation())'</code>
Your code	Python 3.10, system interpreter	—

Baked into the image: `python3-opencv` (GStreamer-enabled), `python3-gi` , `numpy` , `pyserial` , `rich` , `pymavlink` , MAVProxy, `v4l-utils` , `i2c-tools` , `gpiod` .

The ISP, and why it keeps coming up

The IMX477 produces a Bayer mosaic — one colour sample per pixel. Turning that into a picture (demosaic, black level, lens shading, white balance, gamma, noise reduction) is the ISP's job, and on Tegra the ISP is only reachable through **Argus**, a userspace daemon you talk to via `nvarguscamerasrc` .

Argus requires an EGL display. Over plain SSH there is none, so it fails with `Failed to initialize EGLDisplay`. This is not a bug in the tools; it is the platform. Your three options are: run with a real display on the board, export a working `DISPLAY/XAUTHORITY` into your SSH session, or bypass the ISP and accept raw Bayer.

This matters for your own pipeline design too. **If your algorithm can work on raw or minimally-processed frames, it will be simpler and lower-latency to run headless.** If it needs a properly exposed colour image, plan for a display context on the board.

The 15-pin connector is a two-lane connector

The A603 has one 15-pin CSI-2 FPC connector, which carries **two** data lanes, not four. That caps the pixel rate, and therefore the achievable resolution-times-framerate product. Do not assume the sensor's headline modes are available — read the actual mode table:

```
v4l2-ctl -d /dev/video0 --list-formats-ext
```

The `max_framerate` values in that list are the truth for this carrier.

2. Camera programs

All the viewers auto-detect the sensor type and pick a pipeline accordingly: Bayer (IMX477) → `nvarguscamerasrc`; YUV/UYVY modules → `v4l2src`. They all serve MJPEG over HTTP on `--port 8080` unless noted.

live-view.py — the one that works over SSH

Camera to a browser. The simplest viewer, deliberately: it uses `cv2.VideoCapture`, carries **no** timestamps, and has the fewest ways to go wrong. When something else is broken, this is the tool that still runs.

```
./live-view.py [--device /dev/video0] [--port 8080]
               [--width 1920] [--height 1080] [--fps 30]
```

Routes: `/` (viewer page), `/stream` (`multipart/x-mixed-replace` MJPEG).

Over SSH, Argus fails and the script falls back to a software debayer with no AE and no AWB — flat, dim colour. That is the fallback working as designed, not the camera failing.

live-view-pts.py — same, with real capture timestamps

Identical viewer, except each frame carries the time it was **captured** rather than the time Python saw it:

```
absolute capture time = pipeline.base_time + buffer.pts
```

`cv2.VideoCapture` throws the PTS away, so an OpenCV viewer can only stamp arrival — which includes tens of milliseconds of pipeline latency and jitter. This script drives GStreamer directly through `python3-gi` to keep the real value. The pipeline's clock domain is **queried at startup, not assumed**.

Adds `--csv FILE`, logging `frame,pts_ns,mono_s,utc`.

live-view-imu.py — camera and attitude on one page

The camera stream with the flight controller's attitude burned into the JPEG as a horizon indicator, plus a numeric panel fed from a separate JSON endpoint so the digits stay crisp and update faster than the video.

```
./live-view-imu.py --msp /dev/ttyTHS1 [--baud 115200] [--imu-hz 30]
                        [--no-horizon] [--port 8080] [--device /dev/video0]
```

Routes: `/`, `/stream`, `/imu.json`.

The point of burning the horizon into the frame is evidential: a screenshot proves the camera and the IMU were live **at the same instant**, which two separate windows cannot.

The two halves fail independently and visibly. No FC still gives you video with a red panel. The panel distinguishes **STALE** (port open, FC has gone quiet) from **DOWN** (port unusable) — a link that stopped replying must never keep displaying the last attitude as though it were current. That distinction is worth copying into your own code.

show-camera.py — local display, full ISP control

The local-display counterpart to `live-view.py`. Needs HDMI/DP or an X session on the board.

```
./show-camera.py [--sensor-mode N] [--flip 0] [--wb auto]
                  [--exposure-time MIN MAX] [--gain MIN MAX] [--isp-gain MIN MAX]
                  [--saturation 1.0] [--ee-mode 1] [--tnr-mode 1] [--ev 0]
                  [--ae-lock] [--awb-lock] [--allow-cpu-debayer] [--no-osd]
```

OPTION	MEANING
<code>--sensor-mode</code>	Mode index, in <code>--list-formats-ext</code> order
<code>--exposure-time</code>	Two integers, nanoseconds. Equal values pin the exposure
<code>--gain</code> / <code>--isp-gain</code>	Analogue / ISP digital gain range
<code>--ev</code>	Exposure compensation, -2.0 to 2.0
<code>--ae-lock</code> / <code>--awb-lock</code>	Freeze auto-exposure / auto-white-balance
<code>--ee-mode</code> / <code>--tnr-mode</code>	Edge enhance / temporal noise reduction: 0 off, 1 fast, 2 HQ

Keys: `q` / `ESC` quit · `s` save `capture-NNN.png` · `f` fullscreen.

ISP settings are pipeline properties, fixed when the stream starts — OpenCV gives no handle to change them live, so changing one means restarting with different flags. The defaults mean "let the ISP decide", which is usually right; reach for `--exposure-time` and `--gain` only when AE is chasing a scene it cannot win.

If Argus cannot start, the script **stops** rather than quietly showing a software-debayered image that looks like a broken camera.

raw-view.py — sensor truth, not a picture

No ISP, no Argus, no white balance unless you ask. This answers "*is the sensor producing sane data*", never "*does this look nice*".

```
./raw-view.py [--pixelformat RG10] [--bits 10] [--bayer RGGB] [--stride 0]
               [--scale 0] [--gamma 2.2] [--gain 1.0] [--view 2]
               [--sensor-mode N] [--ctrl gain=200 --ctrl exposure=20000]
```

Capture goes through `v4l2-ctl --stream-to=-` rather than GStreamer, because GStreamer's `video/x-bayer` is 8-bit only and cannot carry a 10-bit RG10 stream at all.

Views (number keys): `1` mosaic (raw greyscale, full res — zoom in and the Bayer checkerboard should be visible) · `2` demosaic (exact 2×2 quad average, not interpolated) · `3` planes (R, G1, G2, B side by side) · `4` OpenCV's interpolating demosaic.

Other keys: `a` grey-world WB · `h` histogram · `g`/`G` gamma · `-`/`+` display gain · `c` cycle the Bayer code if colours look swapped · `s` save `frame-NNN.raw` (exact bytes) + `.png` · `p` print the 2×2 quad under the mouse.

`--gamma` and `--gain` are **display only** — they never touch the data you save. There is no auto-exposure here, because AE lives in Argus, which is exactly what this bypasses: set exposure and gain yourself and use the OSD's min/max/mean to judge whether you are under- or over-exposed.

frame-timestamps.py — timing and jitter

Per-frame capture timestamps in a clock domain you can actually align against, with inter-frame deltas. Drives GStreamer directly and reads `GstBuffer.pts`.

```
./frame-timestamps.py          # print to stdout
./frame-timestamps.py --limit 300 --csv frames.csv
./frame-timestamps.py --verify  # prove the clock domain, then exit
./frame-timestamps.py --test-source # videotestsrc, no camera needed
```

What the timestamp actually is: it is latched in hardware at the frame boundary by NVCSI/VI, written by the camera R5 core, and passed through the V4L2 buffer to the GStreamer PTS untouched. CPU scheduling and ISP work move the `latency` column; they never move the timestamp. That is what makes it usable for anything serious.

3. Camera–IMU timing: read this before you build VIO

This is the section most likely to save you a week. Three facts about this hardware, in increasing order of how much trouble they cause:

1. The timestamp is the SoC's clock, not the camera's. The IMX477 never timestamps anything; the Tegra TSC does, on arrival. Sensor and SoC crystals differ by tens of ppm, so a nominal 30.000 fps stream measures about 29.9997 fps and the error **accumulates**. In the `dt` column this shows up as a slow ramp, visibly different from scheduling scatter.

2. It marks a frame boundary in the CSI stream, which on a rolling shutter is *after* the first row's exposure ended. The exposure midpoint you actually want to pair with an IMU sample is **behind** the timestamp, not ahead of it. Verify the SOF/EOF convention on your own hardware before you trust a sign.

3. There is no hardware sync line. The IMU is on the flight controller, in Betaflight's timebase. The 15-pin RPi FPC on this carrier breaks out **no XVS/strobe line**, so the camera physically cannot tell the FC when a frame happened, and the FC cannot trigger the camera. Closing that gap needs one of:

- a shared physical edge (GPS PPS, or an FC-driven GPIO into `nvpps`),
- continuous timesync messages over the link, or
- letting the estimator solve for the offset online — VINS-Mono's `td`, or an offline Kalibr calibration.

The third is what most teams will end up doing. If you calibrate a camera–IMU offset, **pin the exposure first** (`show-camera.py --exposure-time` with equal min and max, plus `--ae-lock`) — otherwise AE moves the effective capture instant every frame and no calibration will hold.

4. Flight controller programs

Everything here lives in `~/target/msp/`. The flight controller runs **Betaflight**, which speaks **MSP** — a request/response protocol over serial. It is not MAVLink. Tools written for ArduPilot or PX4 will not work.

`setup_jetson_uart.sh` — run this first, once per board

By default Linux runs a serial console on `/dev/ttyTHS1`, and it will compete with you for every byte on the line.

```
./setup_jetson_uart.sh          # inspect only, changes nothing
sudo ./setup_jetson_uart.sh --apply  # permissions, console, udev rule
```

`msp_bench.py` — the main entry point

```
python3 msp_bench.py [--port DEV] [--baud 115200] [--timeout 0.5] [--msp-v2] \
    {info|telemetry|rc|demo} [...]
```

`--port`, `--baud`, `--timeout` and `--msp-v2` are **global** options and must come **before** the subcommand. Put them after and `argparse` rejects the line.

SUBCOMMAND	WHAT IT DOES	MOTORS
<code>info</code>	Identify the FC, its sensors, its arming blockers, the battery. Read-only.	still
<code>telemetry</code> <code>--hz 20</code>	Stream attitude / IMU / battery. Read-only.	still
<code>rc</code> <code>--rate 100</code>	Stream RC frames and echo <code>MSP_RC</code> back. Throttle stays at minimum; ARM is never raised.	still
<code>demo</code> <code>--props-off</code>	Arms the FC and runs a throttle/attitude profile.	spin

`demo` takes `--max-throttle 0.15` (a hard clamp, 0-1) and `--max-angle 0.3` (stick authority clamp). It **refuses to start without** `--props-off`, which is an assertion by you — the code cannot verify it. It clears the MSP arming lock, confirms nothing else blocks arming, arms, runs a fixed ~10 s profile (hover, pitch, roll, yaw, idling at 30-50 % of `--max-throttle`), then disarms and re-asserts the arming lock in a `finally` block. Ctrl-C aborts down the same path.

For a slow idle spin, lower the ceiling — `--max-throttle 0.05` gives roughly 1.5-2.5 % actual throttle.

`imu_check.py` — the functional IMU test

Deliberately far more than "did the FC answer". A **dead or wedged IMU very often still returns plausible numbers**: an all-zero gyro looks exactly like a perfectly still one, and a frozen accelerometer buffer holding 1 g looks like a level board.

```
./imu_check.py --dev /dev/ttyTHS1 [--baud 115200] [--samples 200] [--hz 50]
    [--acc-1g 512] [--motion] [--tsv results.tsv] [--json imu.json]
```

CHECK	WHAT IT CATCHES
identity	link down; also reads the MCU's unique hardware serial
FC reports ACC and GYRO	sensor not detected by the firmware at all
sample rate	link too slow to be useful

CHECK	WHAT IT CATCHES
CRC / I ² C error counts	marginal wiring, baud mismatch, bad sensor bus
accel reads 1 g at rest	wrong scaling, dead axis, uncalibrated part
gyro near zero at rest	saturated or drifting gyro
data is changing	a frozen buffer — which passes every check above
attitude ↔ accel cross-check	a mis-mapped axis

`--motion` additionally asks you to rotate the board, and checks that both the gyro and the attitude estimate follow. `--acc-1g` overrides the scale for non-Betaflight firmware (Betaflight: 512 counts = 1 g, gyro already in deg/s; INAV differs). Exit status is 0 only if every check passed, so it drops straight into a script.

companion_listener_msp.py — live telemetry table

A live table with a rate sparkline and stutter/stop detection.

```
# safest first
python3 companion_listener_msp.py --port /dev/ttyTHS1 --listen-only
python3 companion_listener_msp.py --port /dev/ttyTHS1 --pin-throttle
python3 companion_listener_msp.py --port /dev/ttyTHS1          # ramps throttle
```

OPTION	DEFAULT	MEANING
<code>--listen-only</code>	off	decode only, transmit nothing — safe with a live FC
<code>--pin-throttle</code>	off	transmit, but hold throttle at minimum; move only roll/pitch/yaw
<code>--poll-rate</code>	50.0	cap the poll loop, cycles/s; 0 = as fast as the link allows
<code>--slow-divisor</code>	10	poll status/analog/GPS/altitude every Nth cycle
<code>--msp-timeout</code>	0.08	per-request deadline, short so one dead command cannot stall the view
<code>--ramp-period</code>	10.0	seconds for one 1000 → 2000 sweep
<code>--max-throttle</code>	1.0	ceiling for the ramp, 0–1

Because MSP is polled rather than pushed, the health statistics mean something different from a streaming link: frames/s becomes **completed poll cycles/s**, and `crc_errors` counts only real line corruption (MSP frames are addressed by command id, so a nonzero value here means the **wiring or baud rate** is bad, not the software).

Polling cost is not uniform, so the loop splits in two: attitude/IMU/RC every cycle, status/analog/GPS/altitude every Nth. That keeps the fast fields at the highest rate the link allows instead of dragging them down to the slowest.

This tool cannot spin motors. It never clears the arming lock, and AUX1 is part of the RC frame it sends, held low — so streaming actively holds the FC *disarmed* rather than merely not arming it. Betaflight also blocks arming while an MSP client is connected. Neither is a substitute for taking the props off.

5. Writing your own code

Import the same libraries the tools use. They are in `~/target/msp/`, and they are the supported interface — do not re-implement MSP framing.

Frames, via OpenCV

```
import cv2

# ISP path – best image, needs a display/EGL context.
ISP = ("nvarguscamerasrc ! "
      "video/x-raw(memory:NVMM),width=1920,height=1080,framerate=30/1 ! "
      "nvvidconv ! video/x-raw,format=BGRx ! videoconvert ! "
      "video/x-raw,format=BGR ! appsink drop=true max-buffers=1 sync=false")

cap = cv2.VideoCapture(ISP, cv2.CAP_GSTREAMER)
if not cap.isOpened():
    raise SystemExit("pipeline would not open – check EGL, or fall back to v4l2")

while True:
    ok, frame = cap.read()      # frame is a (1080, 1920, 3) BGR ndarray
    if not ok:
        continue
    ...
```

`drop=true max-buffers=1` is not optional for anything real-time: without it GStreamer queues frames and your algorithm falls further behind every second until it is processing seconds-old video.

If `isOpened()` returns `False` with no error text, the usual cause is an OpenCV built without GStreamer — i.e. one installed from pip. Check with `python3 -c "import cv2; print(cv2.getBuildInformation())" | grep -i gstreamer`.

Telemetry, via MSPLink

```
import sys
sys.path.insert(0, "/home/dcL/target/msp")
from msp import MSPLink

with MSPLink("/dev/ttyTHS1", 115200) as fc:
    print(fc.api_version(), fc.fc_variant(), fc.uid())
    roll, pitch, yaw = fc.attitude()      # degrees; yaw 0-360
    acc, gyro, mag    = fc.raw_imu()       # raw FC units
    batt              = fc.analog()        # dict: voltage_v, current_a,
                                          #         mah_drawn, rssi
```

`MSPLink` is thread-safe and runs a background RX loop. `request()` blocks for a reply and raises `MSPTimeout` or `MSPError`; `send()` is fire-and-forget, for cases where latency beats confirmation. The decoder never blocks and never raises on junk — corrupt frames are counted in `crc_errors()` and dropped, so a noisy line degrades your rate rather than crashing your process.

Useful methods: `attitude()`, `raw_imu()`, `altitude()`, `analog()`, `rc_channels()` (the channels the FC is acting on), `motors()`, `status()`, `uid()`, and `decode_arwing_flags()` / `decode_sensor_flags()` to turn bitfields into readable names.

If you need to send RC

`msp_rc.RCTransmitter` streams `MSP_SET_RAW_RC` at a fixed rate from a background thread, with a watchdog: `set_control()` must be called at least every `command_timeout_s` or the transmitter falls back to its idle frame. `arm()` refuses unless throttle is already at minimum, and `close()` disarms, holds the idle frame long enough for the FC to see it, then stops the stream.

Keep those three safety properties if you write your own transmitter. They are there because each one has a failure mode behind it.

Developing with no hardware

`fake_fc.py` simulates a Betaflight FC on a pty, with fault modes `healthy`, `frozen`, `saturated`, `badscale`, `noaccel`, `i2cerr`, `nouid` — each one reproducing a real way an IMU fails *while still talking*.

```
from fake_fc import FakeFC
fc = FakeFC("frozen")      # fc.dev is a pty path you can point any tool at
```

The whole test suite runs with nothing plugged in:

```
cd ~/target/msp && python3 test_msp_loopback.py && python3 test_imu_check.py \
    && python3 test_imu_panel.py && python3 test_companion_listener_msp.py
```

And `frame-timestamps.py --test-source` exercises the camera timing path against `videotestsrc`, with no camera attached.

6. Diagnostics

SCRIPT	QUESTION IT ANSWERS
<code>bringup-check.sh</code>	"Is the whole stack up?" Walks every layer in dependency order and stops at the first hard failure
<code>camera-bind-check.sh</code>	"Why is the camera driver not bound?" Walks the bind chain: DT node → I ² C adapter → I ² C client → driver binding → module → bus response → probe log → platform stack → reset line
<code>usb-device-mode.sh</code>	"Why is 192.168.55.1 not answering?" Checks the USB device controller, role switch, VBUS detect line, the L4T gadget service and <code>l4tbr0</code>
<code>provision.sh</code>	Re-installs the camera and FC tooling on a board that was flashed without it. Needs network. Idempotent

"Driver not bound" is a conclusion, not a cause — it happens when the DT node is absent, when the I²C client is never instantiated, when the module cannot load, when the sensor does not answer on the bus, or when probe runs and fails. Those need completely different fixes, which is why `camera-bind-check.sh` exists rather than a single yes/no test.

update-dtb.sh — do not use it

It is retained for signed-boot setups only and refuses to run without `--i-know`. On this platform, as flashed, writing the `kernel-dtb` partition has **no effect**: UEFI rejects the unsigned partition copy and boots the device tree embedded in the bootloader image instead. A `dd` into that partition verifies byte-for-byte clean and changes nothing — a genuinely misleading failure mode that has already cost one debugging cycle. Changing the device tree requires a full re-flash.

7. Safety rules

1. **Props off before anything that can arm.** Exactly one program can arm the aircraft: `msp_bench.py demo`. It requires `--props-off`, which is your word, not a measurement.
2. `companion_listener_msp.py` with no flags transmits RC with a **throttle ramp**. Start with `--listen-only`.
3. **Never assume a stale reading is a current one.** The MSP tools distinguish LIVE / STALE / DOWN for this reason; carry that distinction into your own code.

4. **Shut down cleanly** — `sudo shutdown -h now`. The rootfs is on an NVMe SSD and pulling power mid-write will eventually corrupt it.
 5. **The Jetson is not a flight controller.** MSP over a 115200-baud serial line, polled, is telemetry and supervision. Anything that must close a loop at flight rates belongs inside Betaflight.
-

8. Known limits of this hardware

Not bugs — constraints to design around:

- **No hardware camera/IMU sync line.** The 15-pin FPC breaks out no XVS or strobe. Camera-IMU offset must be solved in software (§3).
- **Argus needs EGL.** No ISP-processed image over a bare SSH session (§1).
- **Two CSI data lanes,** so the mode table is shorter than the sensor's datasheet suggests (§1).
- **One CSI connector.** A second camera is not an option on this carrier.
- **Device tree changes need a full re-flash** (§6).
- **25 W power budget by default,** and sustained GPU load will thermally throttle. Watch `tegrastats`; `nvpmodel -q` shows the current budget.